Tugas Pertemuan 1	
IF21W0508 – Praktikum Web	
NPM	243040002
Nama	Ardhi Jamil Muchtar
Kelas	E
Link Github Repository	https://github.com/ArdhiJM/PW2026_243040002/tree/main/Pertemuan2

Bagian 1. Screenshot Semua Kode

App.tsx

Pertemuan2 > src > App.tsx > ...

```tsx
import { useState, useEffect } from 'react';
import './App.css';
import Kartu from './Kartu';
import Penghitung from './Penghitung';
import FormTugas from './FormTugas';
import ItemTugas, { type Tugas } from './ItemTugas';
import CatatanKeuangan from './CatatanKeuangan';

function App() {
  const [daftarTugas, setDaftarTugas] = useState<Tugas[]>(() => {
    const dataTersimpan = localStorage.getItem('daftar-tugas');
    if (dataTersimpan) {
      return JSON.parse(dataTersimpan);
    }
    return [
      { id: 1, judul: 'Setup Lingkungan Pengembangan Vite & TypeScript', selesai: true },
      { id: 2, judul: 'Pelajari Hook useState & Controlled Form', selesai: false }
    ];
  });

  const [filter, setFilter] = useState<'semua' | 'belum' | 'selesai'>('semua');

  useEffect(() => {
    localStorage.setItem('daftar-tugas', JSON.stringify(daftarTugas));
  }, [daftarTugas]);

  useEffect(() => {
    const sisa = daftarTugas.filter((t) => !t.selesai).length;
    document.title = `(${sisa}) PErtemuan 2 - Praktikum Web`;
  }, [daftarTugas]);

  const tambahTugas = (judul: string) => {
    const tugasBaru: Tugas = { id: Date.now(), judul: judul, selesai: false };
    setDaftarTugas([...daftarTugas, tugasBaru]);
  };

  const ubahStatus = (id: number) => {
    const daftarBaru = daftarTugas.map((tugas: Tugas) => {
      if (tugas.id === id) {
        return { ...tugas, selesai: !tugas.selesai };
      }
      return tugas;
    });
    setDaftarTugas(daftarBaru);
  };

  const hapusTugas = (id: number) => {
    setDaftarTugas(daftarTugas.filter((tugas: Tugas) => tugas.id !== id));
  };

  const hapusSemuaSelesai = () => {
    setDaftarTugas(daftarTugas.filter((tugas: Tugas) => !tugas.selesai));
  };

  const jumlahSelesai = daftarTugas.filter((tugas: Tugas) => tugas.selesai).length;

  const daftarTugasTersaring = daftarTugas.filter((tugas: Tugas) => {
    if (filter === 'belum') return !tugas.selesai;
    if (filter === 'selesai') return tugas.selesai;
    return true;
  });
```

App.tsx

Pertemuan2 > src > App.tsx > ...

```tsx
function App() {

  return (
    <div className="wadah">
      <header className="app-header">
        <span className="badge-sistem">PRAKTIKUM WEB · MODUL 02</span>
        <h1 className="judul">Praktikum Pemrograman Web</h1>
        <p className="subjudul">
          Ruang kerja praktikum untuk mengelola agenda, memahami state, dan mencatat arus kas dalam satu dashboard.
        </p>
      </header>

      <main className="dashboard-layout">
        <div className="agenda-column">
          {/* Latihan 1: State Interaktif */}
          <Penghitung />

          {/* Latihan 2 & Tugas 1-2: Aplikasi Pengelola Agenda Tugas */}
          <Kartu judul="Tambah Agenda Tugas Baru">
            <FormTugas onTambah={tambahTugas} />
          </Kartu>

          <Kartu judul="Daftar Agenda Sesi Praktikum">
            <div className="ringkasan-box">
              <div className="ringkasan-teks">
                <span>Progres penyelesaian</span>
                <strong>{jumlahSelesai} <span className="ringkasan-total">/ {daftarTugas.length}</span></strong>
              </div>
              <div
                className="progress-trek"
                role="progressbar"
                aria-label="Progres penyelesaian tugas"
                aria-valuenow={daftarTugas.length === 0 ? 0 : Math.round((jumlahSelesai / daftarTugas.length) * 100)}
                aria-valuemin={0}
                aria-valuemax={100}
              >
                <span
                  className="progress-isi"
                  style={{ width: `${daftarTugas.length === 0 ? 0 : (jumlahSelesai / daftarTugas.length) * 100}%` }}
                />
              </div>
            </div>

            {/* Tombol Filter */}
            <div className="grup-filter" aria-label="Filter agenda">
              <button
                className={`tombol-filter ${filter === 'semua' ? 'aktif' : ''}`}
                onClick={() => setFilter('semua')}
              >
                Semua ({daftarTugas.length})
              </button>
              <button
                className={`tombol-filter ${filter === 'belum' ? 'aktif' : ''}`}
                onClick={() => setFilter('belum')}
              >
                Belum Selesai ({daftarTugas.length - jumlahSelesai})
              </button>
              <button
                className={`tombol-filter ${filter === 'selesai' ? 'aktif' : ''}`}
                onClick={() => setFilter('selesai')}
              >
                Selesai ({jumlahSelesai})
              </button>
            </div>

            {daftarTugasTersaring.length === 0 ? (
              <p className="kosong">Tidak ada agenda tugas yang sesuai dengan filter saat ini.</p>
            ) : (
              <ul className="daftar">
                {daftarTugasTersaring.map((tugas: Tugas) => (
                  <ItemTugas
                    key={tugas.id}
                    tugas={tugas}
                    onUbahStatus={ubahStatus}
                    onHapus={hapusTugas}
                  />
                ))}
              </ul>
            )}

            {jumlahSelesai > 0 && (
              <div className="area-aksi-bawah">
                <button
                  className="tombol tombol-bahaya tombol-kecil"
                  onClick={hapusSemuaSelesai}
                >
                  Hapus Semua yang Selesai ({jumlahSelesai})
                </button>
              </div>
            )}
          </Kartu>
        </div>

        {/* Tugas 3: Aplikasi Catatan Keuangan Mini */}
        <CatatanKeuangan />
      </main>
    </div>
  );
}

export default App;
```

CatatanKeuangan.tsx

Pertemuan2 > src > CatatanKeuangan.tsx > ...

```tsx
import { useState, useEffect } from 'react';
import Kartu from './Kartu';
import FormTransaksi, { type DataTransaksiBaru } from './FormTransaksi';
import ItemTransaksi, { type Transaksi } from './ItemTransaksi';

function CatatanKeuangan() {
  const [daftarTransaksi, setDaftarTransaksi] = useState<Transaksi[]>(() => {
    const dataTersimpan = localStorage.getItem('catatan-keuangan');
    if (dataTersimpan) {
      return JSON.parse(dataTersimpan);
    }
    return [
      { id: 1, keterangan: 'Alokasi Dana Praktikum', nominal: 500000, jenis: 'pemasukan' },
      { id: 2, keterangan: 'Pembelian Buku Referensi', nominal: 125000, jenis: 'pengeluaran' }
    ];
  });

  useEffect(() => {
    localStorage.setItem('catatan-keuangan', JSON.stringify(daftarTransaksi));
  }, [daftarTransaksi]);

  const handleTambahTransaksi = (dataBaru: DataTransaksiBaru) => {
    const transaksiBaru: Transaksi = { id: Date.now(), ...dataBaru };
    setDaftarTransaksi([transaksiBaru, ...daftarTransaksi]);
  };

  const handleHapusTransaksi = (id: number) => {
    setDaftarTransaksi(daftarTransaksi.filter((t: Transaksi) => t.id !== id));
  };

  const totalPemasukan = daftarTransaksi
    .filter((t: Transaksi) => t.jenis === 'pemasukan')
    .reduce((acc: number, curr: Transaksi) => acc + curr.nominal, 0);

  const totalPengeluaran = daftarTransaksi
    .filter((t: Transaksi) => t.jenis === 'pengeluaran')
    .reduce((acc: number, curr: Transaksi) => acc + curr.nominal, 0);

  const totalSaldo = totalPemasukan - totalPengeluaran;

  return (
    <section className="keuangan-column">
      <div className="section-header">
        <span className="section-kicker">ADMINISTRASI</span>
        <h2>Aplikasi Catatan Keuangan Mini</h2>
        <p>Pencatatan arus kas operasional berbasis penyimpanan lokal (localStorage).</p>
      </div>

      <div className="dashboard-grid">
        <div className="stat-card stat-saldo">
          <span className="stat-label">Total Saldo Kas</span>
          <h3 className={`stat-value ${totalSaldo >= 0 ? 'teks-netral' : 'teks-pengeluaran'}`}>
            Rp {totalSaldo.toLocaleString('id-ID')}
          </h3>
        </div>
        <div className="stat-card stat-pemasukan">
          <span className="stat-label">Total Pemasukan</span>
          <h3 className="stat-value teks-pemasukan">
            + Rp {totalPemasukan.toLocaleString('id-ID')}
          </h3>
        </div>
```

CatatanKeuangan.tsx

Pertemuan2 > src > CatatanKeuangan.tsx > ...

```tsx
function CatatanKeuangan() {
            <div className="stat-card stat-pengeluaran">
              <span className="stat-label">Total Pengeluaran</span>
              <h3 className="stat-value teks-pengeluaran">
                - Rp {totalPengeluaran.toLocaleString('id-ID')}
              </h3>
            </div>
          </div>

          <Kartu judul="Tambah Transaksi Baru">
            <FormTransaksi onTambah={handleTambahTransaksi} />
          </Kartu>

          <Kartu judul="Riwayat & Audit Transaksi">
            {daftarTransaksi.length === 0 ? (
              <p className="kosong">Belum ada riwayat transaksi yang tercatat.</p>
            ) : (
              <ul className="daftar">
                {daftarTransaksi.map((transaksi: Transaksi) => (
                  <ItemTransaksi
                    key={transaksi.id}
                    transaksi={transaksi}
                    onHapus={handleHapusTransaksi}
                  />
                ))}
              </ul>
            )}
          </Kartu>
        </section>
      );
    }

    export default CatatanKeuangan;
```

FormTransaksi.tsx

Pertemuan2 > src > FormTransaksi.tsx > ...

```tsx
import { useState, type FormEvent } from 'react';

export interface DataTransaksiBaru {
  keterangan: string;
  nominal: number;
  jenis: 'pemasukan' | 'pengeluaran';
}

interface FormTransaksiProps {
  onTambah: (data: DataTransaksiBaru) => void;
}

function FormTransaksi({ onTambah }: FormTransaksiProps) {
  const [keterangan, setKeterangan] = useState<string>('');
  const [nominal, setNominal] = useState<string>('');
  const [jenis, setJenis] = useState<'pemasukan' | 'pengeluaran'>('pemasukan');

  const tidakValid = keterangan.trim() === '' || nominal === '' || Number(nominal) <= 0;

  const handleSubmit = (e: FormEvent) => {
    e.preventDefault();
    if (tidakValid) return;

    onTambah({
      keterangan: keterangan.trim(),
      nominal: Number(nominal),
      jenis: jenis
    });

    setKeterangan('');
    setNominal('');
    setJenis('pemasukan');
  };

  return (
    <form onSubmit={handleSubmit}>
      <div className="form-group">
        <label className="label-input">Keterangan Transaksi</label>
        <input
          className="input"
          type="text"
          placeholder="Contoh: Pembelian Modul, Honor Asisten"
          value={keterangan}
          onChange={(e) => setKeterangan(e.target.value)}
        />
      </div>

      <div className="baris-ganda">
        <div className="form-group flex-1">
          <label className="label-input">Nominal (Rp)</label>
          <input
            className="input"
            type="number"
            placeholder="0"
            value={nominal}
            onChange={(e) => setNominal(e.target.value)}
          />
        </div>

        <div className="form-group flex-1">
          <label className="label-input">Jenis Transaksi</label>
          <select
            className="input select-input"
            value={jenis}
            onChange={(e) => setJenis(e.target.value as 'pemasukan' | 'pengeluaran')}
          >
            <option value="pemasukan">Pemasukan (+)</option>
            <option value="pengeluaran">Pengeluaran (-)</option>
          </select>
        </div>
      </div>

      <button className="tombol tombol-utama lebar-penuh" type="submit" disabled={tidakValid}>
        + Catat Transaksi
      </button>
    </form>
  );
}

export default FormTransaksi;
```

ItemTransaksi.tsx

Pertemuan2 > src > ItemTransaksi.tsx > Transaksi

```tsx
export interface Transaksi {
  id: number;
  keterangan: string;
  nominal: number;
  jenis: 'pemasukan' | 'pengeluaran';
}

interface ItemTransaksiProps {
  transaksi: Transaksi;
  onHapus: (id: number) => void;
}

function ItemTransaksi({ transaksi, onHapus }: ItemTransaksiProps) {
  const isPemasukan = transaksi.jenis === 'pemasukan';

  return (
    <li className="item-transaksi">
      <div className="info-transaksi">
        <strong className="judul-transaksi">{transaksi.keterangan}</strong>
        <span className={`badge-jenis ${isPemasukan ? 'badge-masuk' : 'badge-keluar'}`}>
          {transaksi.jenis}
        </span>
      </div>

      <div className="aksi-transaksi">
        <span className={`nominal-transaksi ${isPemasukan ? 'teks-pemasukan' : 'teks-pengeluaran'}`}>
          {isPemasukan ? '+' : '-'} Rp {transaksi.nominal.toLocaleString('id-ID')}
        </span>

        <button
          className="tombol tombol-bahaya tombol-kecil"
          onClick={() => onHapus(transaksi.id)}
        >
          Hapus
        </button>
      </div>
    </li>
  );
}

export default ItemTransaksi;
```

ItemTugas.tsx

Pertemuan2 > src > ItemTugas.tsx > ...

```tsx
export type Tugas = {
    id: number;
    judul: string;
    selesai: boolean;
};


type ItemTugasProps = {
    tugas: Tugas;
    onUbahStatus: (id: number) => void;
    onHapus: (id: number) => void;
};

function ItemTugas({ tugas, onUbahStatus, onHapus }: ItemTugasProps) {
    return (
        <li className="item">
            <input
                type="checkbox"
                checked={tugas.selesai}
                onChange={() => onUbahStatus(tugas.id)}
            />
            {/* className dinamis: dicoret jika tugas sudah selesai */}
            <span className={tugas.selesai ? 'selesai' : ''}>{tugas.judul}</span>
            <button
                className="tombol tombol-merah tombol-kecil"
                onClick={() => onHapus(tugas.id)}
            >
                Hapus
            </button>
        </li>
    );
}
export default ItemTugas;
```

Kartu.tsx

Pertemuan2 > src > Kartu.tsx > ...

```tsx
import type { ReactNode } from 'react';

type KartuProps = {
  judul: string;
  children: ReactNode;
  className?: string;
};

function Kartu({ judul, children, className }: KartuProps) {
  return (
    <section className={`kartu${className ? ` ${className}` : ''}`}>
      <h2>{judul}</h2>
      {children}
    </section>
  );
}

export default Kartu;
```

Penghitung.tsx

Pertemuan2 > src > Penghitung.tsx > ...

```tsx
import { useState } from 'react';
import Kartu from './Kartu';

function Penghitung() {
  const [jumlah, setJumlah] = useState(0);

  const tambah = () => {
    setJumlah(jumlah + 1);
  };

  const ulangi = () => {
    setJumlah(0);
  };

  return (
    <Kartu judul="Pemanasan Klik" className="kartu-penghitung">
      <p className="penghitung-teks">
        Tombol sudah diklik <strong>{jumlah}</strong> kali
      </p>
      {jumlah >= 5 && (
        <p className="pesan-semangat">Wah, kamu rajin sekali mengklik!</p>
      )}
      <div className="baris baris-penghitung">
        <button className="tombol" onClick={tambah}>
          Klik Saya
        </button>
        <button className="tombol tombol-merah" onClick={ulangi}>
          Ulangi
        </button>
      </div>
    </Kartu>
  );
}

export default Penghitung;
```

Bagian 2. Penjelasan Kode

1. App.tsx (Komponen Utama & Management State Agenda Tugas)

- **Inisialisasi State Berbasis Memory Persistabel:** State daftarTugas diinisialisasi menggunakan teknik *lazy initial state* useState<Tugas[]>(() => { ... }). Kode terlebih dahulu mengecek ketersediaan data pada localStorage dengan kunci 'daftar-tugas'. Jika ada, data JSON diparsing; jika kosong, nilai *default* berupa dua item tugas awal akan digunakan.
- **Efek Samping Sinkronisasi (useEffect):**

    - **Efek 1:** Menyimpan array daftarTugas ke localStorage dalam bentuk *string* JSON setiap kali terjadi pembaruan data pada dependensi [daftarTugas].
    - **Efek 2:** Mengubah judul dokumen tab browser (document.title) secara dinamis untuk menampilkan jumlah tugas yang belum selesai.
- **Prinsip *Immutable State Update*:**
    - tambahTugas: Memanfaatkan *spread operator* [...daftarTugas, tugasBaru] untuk menambahkan tugas baru tanpa memutasi array lama.
    - ubahStatus: Menggunakan .map() untuk menghasilkan array baru di mana status selesai dari objek yang cocok diinversikan (!tugas.selesai).
    - hapusTugas & hapusSemuaSelesai: Menggunakan .filter() untuk menyaring item berdasarkan syarat ID atau status penyelesaian.
- **Nilai Turunan (*Derived State*) & Filtering:**jumlahSelesai dan daftarTugasTersaring dihitung langsung secara dinamis saat proses *render* berdasarkan pilihan state filter ('semua', 'belum', 'selesai'), sehingga tidak memerlukan state tambahan yang redundan.

2. CatatanKeuangan.tsx (Pengelolaan Kas & Agregasi Data)

- **Sinkronisasi localStorage:** Mengelola state daftarTransaksi ber-tipe Transaksi[] yang otomatis tersimpan pada kunci 'catatan-keuangan'.
- **Kalkulasi Keuangan Otomatis:**
    - totalPemasukan & totalPengeluaran: Memadukan fungsi .filter() berdasarkan tipe transaksi dan .reduce((acc, curr) => acc + curr.nominal, 0) untuk menjumlahkan nominal transaksi secara fungsional.
    - totalSaldo: Menghitung selisih antara akumulasi pemasukan dan pengeluaran.
- **Formatting Standar:** Menggunakan fungsi JavaScript .toLocaleString('id-ID') untuk memformat tampilan angka nominal menjadi format baku mata uang Rupiah.

3. FormTransaksi.tsx (Controlled Component & Validasi)

- **State Terikat (*Controlled Form*):** Field keterangan, nominal, dan jenis masing-masing diikat ke state lokal menggunakan atribut value dan onChange.
- **Validasi Real-Time:** Variabel tidakValid mengecek kondisi keterangan.trim() === '' || nominal === '' || Number(nominal) <= 0. Jika syarat tidak terpenuhi, tombol *submit* otomatis dinonaktifkan (disabled={tidakValid}).
- **Lifting State Up:** Saat formulir dikirim via onSubmit, e.preventDefault() memblokir aksi *reload* peramban, lalu fungsi callback onTambah dipanggil untuk meneruskan data transaksi baru ke komponen induk.

4. ItemTransaksi.tsx & ItemTugas.tsx (Item Modular & Type Safety)

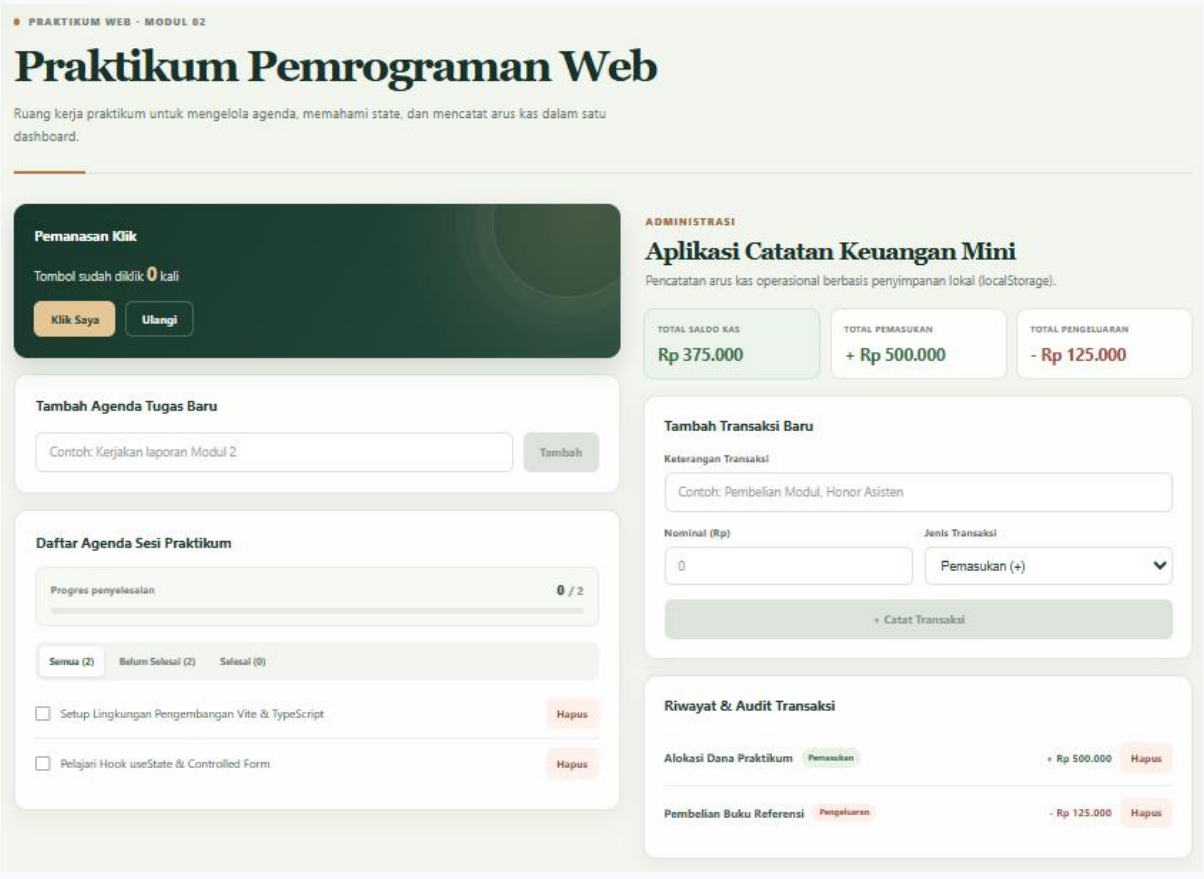
- **TypeScript Interface/Type:** Mendefinisikan kontrak tipe data (Tugas, ItemTugasProps, Transaksi, ItemTransaksiProps) untuk memastikan keamanan tipe (*type safety*) pada props yang diterima.
- **Pengkondisian Gaya Dinamis:**

- On ItemTugas: Mengaplikasikan kelas CSS dinamis tugas.selesai ? 'selesai' : '' untuk memberikan efek coretan garis tengah pada tugas yang selesai.
- On ItemTransaksi: Menggunakan ternari untuk menentukan kelas pewarnaan badge (badge-masuk / badge-keluar) dan nominal angka (teks-pemasukan / teks-pengeluaran).

5. Kartu.tsx & Penghitung.tsx (Wrapper Component & Basic State)

- **Kartu.tsx**: Menggunakan ReactNode pada props children untuk membuat bingkai kartu pembungkus (*container component*) yang dapat dipakai ulang di berbagai bagian aplikasi.
- **Penghitung.tsx**: Mendemonstrasikan fungsionalitas dasar useState untuk mencatat jumlah klik pengguna dan menam

Bagian 3. Screenshot Hasil Antarmuka



Bagian 4. Penjelasan Hasil Antarmuka

1. Interaktivitas Pengelola Tugas: Pengguna dapat menginputkan judul tugas pada bidang teks. Apabila masukan kosong atau melebihi 50 karakter, tombol masukan

secara otomatis dinonaktifkan (*disabled*). Penekanan tombol filter ("Semua", "Belum Selesai", "Selesai") secara reaktif menyaring item yang ditampilkan sesuai kriteria.

2. Respon Visual & Kondisional Rendering: Pada daftar tugas, centang status mengaktifkan gaya teks tercoret (*line-through*). Tombol "Hapus Semua yang Selesai" hanya akan muncul (*conditional rendering*) apabila terdapat minimal satu tugas berstatus selesai.

3. Dasbor Keuangan Mini: Dasbor keuangan menampilkan ringkasan kartu statistik secara otomatis. Nominal pemasukan diberi highlight latar hijau (+ Rp) dan pengeluaran diberi latar merah (- Rp). Perhitungan Total Saldo dilakukan secara real-time berdasarkan akumulasi transaksi.

4. Persistensi Data (LocalStorage): Seluruh penambahan, pengubahan status, maupun penghapusan data pada aplikasi Daftar Tugas dan Catatan Keuangan tersimpan secara permanen di memori browser. Saat halaman di-*refresh* (F5), seluruh data antarmuka dipulihkan secara utuh tanpa ada informasi yang hilang.